

seus conjuntos pais. Elementos de conjuntos matemáticos não são ordenados, então o modelo não é perfeito.

Note que ponteiros, definidos com operadores de tipos, como `| ast |` em C, não são definidos em termos de uma operação de conjunto.

Isso conclui nossa discussão sobre formalismos em tipos de dados, assim como nossa discussão completa sobre tipos de dados.

RESUMO

Os tipos de dados de uma linguagem influenciam muito a determinação de seu estilo e sua utilidade. Com as estruturas de controle, eles formam a essência de uma linguagem.

Os tipos de dados primitivos da maioria das linguagens imperativas incluem os tipos numéricos, de caracteres e booleanos. Os tipos numéricos, em geral, são diretamente suportados pelo hardware.

Os tipos de enumeração e de subfaixa definidos pelo usuário são convenientes e melhoram a legibilidade e a confiabilidade dos programas.

Matrizes fazem parte da maioria das linguagens de programação. O relacionamento entre uma referência a um elemento de matriz e o endereço desse elemento é dado em uma função de acesso, que é uma implementação de um mapeamento. As matrizes podem ser estáticas, como as de C++, cuja definição inclui o especificador **static**; dinâmicas da pilha fixas, como em funções de C (sem o especificador **static**); dinâmicas do monte fixas, como em objetos Java; ou dinâmicas do monte, como nas matrizes em Perl. A maioria das linguagens permite poucas operações em matrizes completas.

Atualmente, os registros são incluídos na maioria das linguagens. Campos de registros são especificados de diversas maneiras. No caso de COBOL, podem ser referenciados sem a nomeação de todos os registros que os envolvem, apesar de isso ser confuso de se implementar e de prejudicar a legibilidade. Em diversas linguagens que suportam programação orientada a objetos, os registros são suportados com objetos.

Tuplas são semelhantes aos registros, mas não têm nomes para suas partes constituintes. Elas fazem parte de Python, ML e F#.

Listas são fundamentais nas linguagens de programação funcionais, mas atualmente também são incluídas em Python e C#.

Unões são estruturas que podem armazenar valores de tipos diferentes em momentos diferentes. Unões discriminadas incluem uma etiqueta para gravar o valor de tipo atual. Uma união livre é uma união que não tem essa etiqueta. A maioria das linguagens com uniões não tem projetos seguros para elas, com exceção das linguagens ML e F#.

Ponteiros são usados para lidar com a flexibilidade e para controlar o gerenciamento de armazenamento dinâmico. Os ponteiros apresentam alguns perigos inerentes: ponteiros soltos são difíceis de ser evitados, e podem ocorrer vazamentos de memória.

Tipos de referência, como os de Java e C#, fornecem gerenciamento do monte sem os perigos dos ponteiros.

Tipos de enumeração e de registro são relativamente fáceis de implementar. Matrizes também são diretas, apesar de o acesso a seus elementos ser um processo dispendioso quando elas têm diversos índices. A função de acesso requer uma adição e uma multiplicação adicionais para cada índice.

Os ponteiros são relativamente fáceis de implementar se o gerenciamento do monte não for considerado. Esse gerenciamento é simples se todas as células forem do mesmo tamanho, mas é complicado para a alocação e a liberação de células de tamanho variável.

Tipagem forte é o conceito de exigir que todos os erros de tipos sejam detectados. O valor da tipagem forte é um aumento na confiabilidade.

As regras de equivalência de tipos determinam quais operações são válidas entre os tipos estruturados de uma linguagem. As equivalências de tipos por nome e por estrutura são as duas abordagens fundamentais para definir equivalência de tipos.

Teorias de tipos têm sido desenvolvidas em muitas áreas. Na ciência da computação, o ramo prático da teoria de tipos define os tipos e as regras dos tipos das linguagens de programação. A teoria de conjuntos pode ser usada para modelar a maioria dos tipos de dados estruturados nas linguagens de programação.

NOTAS BIBLIOGRÁFICAS

Existe uma literatura abundante que trata do projeto, do uso e da implementação de tipos de dados. Hoare fornece uma das primeiras definições sistemáticas de tipos estruturados em Dahl et al. (1972). Uma discussão geral sobre uma ampla variedade de tipos de dados aparece em Cleaveland (1986).

A implementação de verificações em tempo de execução nas possíveis inseguranças dos tipos de dados em Pascal é discutida em Fischer e LeBlanc (1980). A maioria dos livros de projeto de compiladores, como Fischer e LeBlanc (1991) e Aho et al. (1986), descreve métodos de implementação para tipos de dados, assim como fazem outros textos sobre linguagens de programação, como Pratt e Zelkowitz (2001) e Scott (2009). Uma discussão detalhada sobre os problemas de gerenciamento do monte pode ser encontrada em Tenenbaum et al. (1990). Métodos de coleta de lixo são desenvolvidos por Schorr e Waite (1967) e por Deutsch e Bobrow (1976). Uma lista abrangente de algoritmos de coleta de lixo pode ser encontrada em Cohen (1981) e Wilson (2005).

QUESTÕES DE REVISÃO

1. O que é um descritor?
2. Quais são as vantagens e as desvantagens dos tipos de dados decimais?
3. Quais são as questões de projeto para tipos de cadeias de caracteres?
4. Descreva as três opções para tamanhos de cadeias.
5. Defina *tipos ordinais*, *de enumeração* e *de subfaixa*.
6. Quais são as vantagens dos tipos de enumeração definidos pelo usuário?
7. De que maneira os tipos de enumeração definidos pelo usuário de C# são mais confiáveis que os de C++?
8. Quais são as questões de projeto para matrizes?

9. Defina *matrizes estáticas*, *dinâmicas da pilha fixas*, *dinâmicas do monte fixas* e *dinâmicas do monte*. Quais são as vantagens de cada uma delas?
10. O que acontece quando um elemento não existente de uma matriz é referenciado em Perl?
11. Como JavaScript suporta matrizes esparsas?
12. Que linguagens suportam índices negativos?
13. Que linguagens suportam fatias de matrizes com tamanhos de passos (*stepsizes*)?
14. O que é uma constante agregada?
15. Defina *ordem principal de linha* e *ordem principal de coluna*.
16. O que é uma função de acesso para uma matriz?
17. Quais são as entradas exigidas em um descritor de matriz em Java e quando elas devem ser armazenadas (em tempo de compilação ou em tempo de execução)?
18. Qual é a estrutura de uma matriz associativa?
19. Qual é o propósito dos números de níveis em registros em COBOL?
20. Defina *referências completamente qualificadas* e *elípticas* para campos em registros.
21. Qual é a principal diferença entre um registro e uma tupla?
22. As tuplas de Python são mutáveis?
23. Qual é a finalidade de um padrão de tuplas de F#?
24. Em qual linguagem imperativa as listas servem como matrizes?
25. Qual é a ação da função `CAR` de Scheme?
26. Qual é a ação da função `tl` de F#?
27. De que maneira a função `CDR` de Scheme modifica seu parâmetro?
28. No que se baseiam as compreensões de lista de Python?
29. Defina *união*, *união livre* e *união discriminada*.
30. As uniões de F# são discriminadas?
31. Quais são as questões de projeto para os tipos ponteiro?
32. Quais são os dois problemas mais comuns dos ponteiros?
33. Por que os ponteiros da maioria das linguagens são restritos de forma a apontarem para uma única variável de tipo?
34. O que é um tipo de referência em C++ e para que ele é comumente usado?
35. Por que as variáveis de referência em C++ são melhores que os ponteiros para parâmetros formais?
36. Quais vantagens as variáveis de tipo de referência em Java e C# têm em relação aos ponteiros de outras linguagens?

37. Descreva a abordagem preguiçosa e a ansiosa para recuperar lixo.
38. Por que a aritmética de referências em Java e C# não faz sentido?
39. O que é um tipo compatível?
40. Defina erro de tipo.
41. Defina uma linguagem fortemente tipada.
42. Por que Java não é fortemente tipada?
43. O que é uma conversão implícita sem conversão?
44. Quais linguagens não têm coerções de tipo?
45. Por que C e C++ não são fortemente tipadas?
46. O que é a equivalência de tipos por nome?
47. O que é a equivalência de tipos por estrutura?
48. Qual é a principal vantagem da equivalência de tipos por nome?
49. Qual é a principal desvantagem da equivalência de tipos por estrutura?
50. Para quais tipos C usa a equivalência de tipos por estrutura?
51. Que operação de conjunto modela o tipo de dados **struct** de C?

PROBLEMAS

1. Quais são os argumentos a favor e contra a representação de valores booleanos como bits únicos em memória?
2. Como um valor decimal perde espaço em memória?
3. Os minicomputadores VAX usam um formato para números de ponto flutuante que não é o mesmo do padrão IEEE. Qual é o formato e por que ele foi escolhido pelos projetistas dos computadores VAX? Uma referência para representações de ponto flutuante em VAX é Sebesta (1991).
4. Compare os métodos de lápides e fechaduras e chaves para evitar ponteiros soltos, do ponto de vista da segurança e do custo de implementação.
5. Que desvantagens existem no desreferenciamento implícito de ponteiros, mas apenas em certos contextos?
6. Explique todas as diferenças entre os subtipos e os tipos derivados em Ada.
7. Que justificativa significativa existe para o operador `->` em C e C++?
8. Quais são as diferenças entre os tipos de enumeração de C++ e os de Java?
9. Matrizes multidimensionais podem ser armazenadas em ordem principal de linha, como em C++, ou em ordem principal de coluna, como em Fortran. Desenvolva a função de acesso para ambas as disposições para matrizes tridimensionais.

10. Na linguagem Burroughs Extended ALGOL, as matrizes são armazenadas como uma matriz de uma dimensão de ponteiros para suas linhas, as quais são tratadas como matrizes de uma dimensão de valores. Quais são as vantagens e desvantagens de tal esquema?
11. Analise e escreva uma comparação das funções `malloc` e `free` de C com os operadores **new** e **delete** de C++. Use a segurança como principal consideração na comparação.
12. Analise e escreva uma comparação do uso de ponteiros C++ com o de variáveis de referência em Java para referenciar variáveis dinâmicas do monte fixas. Use a segurança como principal consideração na comparação.
13. Redija um texto curto sobre o que foi perdido e o que foi ganho na decisão dos projetistas de Java de não incluírem os ponteiros de C++.
14. Quais são os argumentos a favor e contra a recuperação de armazenamento do monte implícita de Java, quando comparada com a recuperação de armazenamento do monte explícita exigida em C++? Considere sistemas de tempo real.
15. Quais são os argumentos para a inclusão dos tipos de enumeração em C#, apesar de não estarem disponíveis nas primeiras versões de Java?
16. Qual é sua expectativa em relação ao nível de uso de ponteiros em C#? Com que frequência eles serão usados quando não forem absolutamente necessários?
17. Crie duas listas de aplicações de matrizes, uma para aquelas que exigem matrizes irregulares e outra para aquelas que exigem matrizes retangulares. Agora, discuta se apenas matrizes irregulares, apenas matrizes regulares ou ambas devem ser incluídas em uma linguagem de programação.
18. Compare as capacidades de manipulação de cadeias das bibliotecas de classes de C++, Java e C#.
19. Pesquise a definição de *fortemente tipada*, conforme dada por Gehani (1983), e compare-a com a definição apresentada neste capítulo. De que forma elas são diferentes?
20. De que forma a verificação de tipos estática é melhor que a verificação de tipos dinâmica?
21. Explique como as regras de coerção podem enfraquecer o efeito benéfico da tipagem forte.

EXERCÍCIOS DE PROGRAMAÇÃO

1. Projete um conjunto simples de programas de teste para determinar as regras de compatibilidade de tipos de um compilador C ao qual você tenha acesso. Escreva um relatório sobre suas descobertas.